

# Report - Version 6: Recurrent Language Model

## Introduction

Version 6 extends the TensorFlow neural character language model from Version 5 with a simple recurrent neural network and a hidden state that carries information through an ordered context.

The corpus, vocabulary, four-character context windows, learned embeddings, training and validation split, shuffled mini-batches, cross-entropy objective, automatic differentiation, and autoregressive sampling remain in place. Instead of flattening the four context embeddings into one vector, the model processes them one character at a time through a recurrent hidden state.

```
Characters
↓
Integer identifiers
↓
4-character context
↓
Trainable embeddings
↓
Recurrent hidden states
↓
Output weights
↓
Generated text
```

---

## Goal of Version 6

The model should be able to:

- reuse the same 440-character corpus, 27-character vocabulary, and four-character context windows from Version 5;
- reuse trainable 8-dimensional character embeddings;
- process the four embeddings sequentially instead of flattening them;
- carry information through a 16-dimensional recurrent hidden state;
- learn embedding, input-to-hidden, hidden-to-hidden, and hidden-to-output weights with `tf.GradientTape`;
- preserve mini-batch training, validation monitoring, and reproducible autoregressive generation;
- verify recurrent states, model shapes, gradients, parameter updates, deterministic retraining, and generation.

## Architecture

The complete Version 6 flow is:

```
Corpus: 440 characters
↓
Vocabulary: 27 characters
↓
436 four-character context-target examples
↓
Integer input tensor: (436, 4)
↓
Embedding matrix: (27, 8)
↓
Sequential recurrent processing
↓
Final hidden state: 16 values
↓
Output weight matrix: (16, 27)
↓
Reproducible 80 / 20 example split
↓
GradientTape mini-batch updates
↓
Sliding-window autoregressive generator
```

The context still contains four previous characters, but their embeddings are no longer concatenated into a flat representation. Version 6 processes the context from left to right and reuses the same recurrent weights at every position.

---

## Context windows, split and mini-batches

Sliding windows create the same 436 input-target examples as Version 5. Each example contains four ordered character identifiers. The hidden state starts from zeros for every context example and is updated once for each character.

```
training examples = 348
validation examples = 88
batch size = 32
updates per epoch = 11
```

Only training examples are shuffled and used for parameter updates. Validation examples measure the loss without modifying the model. Neighboring context windows still overlap, so the example-level random split retains the same intentional small-dataset limitation as Version 5.

## Trainable embeddings and recurrent weights

Each character identifier selects one row from the trainable embedding matrix. The four selected vectors are processed sequentially. At every position, the current embedding and the previous hidden state are combined and passed through tanh to produce the next hidden state.

```
Input tensor shape: (436, 4)
Embedding matrix shape: (27, 8)
Embedded context shape: (1, 4, 8)
Input weight matrix shape: (8, 16)
Recurrent weight matrix shape: (16, 16)
Final hidden state shape: (1, 16)
Output weight matrix shape: (16, 27)
Trainable parameters: 1032
```

The embedding matrix contains 216 values, the input-to-hidden matrix 128, the recurrent matrix 256, and the output matrix 432. The model therefore has 1032 trainable parameters, slightly fewer than the 1080 parameters used in Version 5.

---

## Stable TensorFlow loss

The context identifiers are mapped to embeddings and processed recurrently. The final hidden state is multiplied by the output weights to produce next-character logits. Sparse softmax cross-entropy measures the prediction error directly from the logits and integer target IDs.

```
hidden_state, logits = recurrent_forward(
    embedding_matrix, input_weights,
    recurrent_weights, output_weights, inputs
)
example_losses = tf.nn.sparse_softmax_cross_entropy_with_logits(
    labels=targets, logits=logits
)
loss = tf.reduce_mean(example_losses)
```

The initial train and validation losses are both about 3.2958, consistent with an almost uniform prediction over the 27-character vocabulary.

---

## Controlled execution and reproducibility

The notebook keeps CPU execution and suppresses low-level TensorFlow logs. Explicit seeds control TensorFlow parameter initialization, NumPy data splitting, mini-batch shuffling, and text sampling. Repeated runs are deterministic within the same software environment.

## Automatic differentiation

For each mini-batch, `tf.GradientTape` records the recurrent forward calculations and derives the loss with respect to all four trainable parameter matrices:

- `tf.GradientTape` records embedding lookup, recurrent hidden-state updates, output projection, and loss;
- `tape.gradient` calculates gradients for the embedding, input, recurrent, and output weights;
- the sparse embedding gradient is converted to a dense tensor for this small vocabulary;
- `assign_sub` applies gradient descent updates to all four variables;
- training and validation loss are measured after every epoch.

```
with tf.GradientTape() as tape:
    batch_loss = calculate_loss(...)
    embedding_gradient, input_gradient, recurrent_gradient, output_gradient = \
        tape.gradient(batch_loss, [embedding_matrix, input_weights,
                                   recurrent_weights, output_weights])
```

Training configuration:

```
Learning rate: 1.0
Batch size: 32
Epochs: 100
Seed: 42
```

## Training result

```
Initial train loss: 3.295837
Final train loss: 1.525101
Initial validation loss: 3.295840
Final validation loss: 3.035164
Best validation loss: 2.839605
Best validation epoch: 92
```

The recurrent model learns slowly during the first part of training. After roughly 50 epochs, training loss begins to decrease clearly and continues to improve through epoch 100. Validation loss also improves and reaches its minimum at epoch 92.

After the best epoch, validation loss increases slightly while training loss continues to fall. This shows the beginning of overfitting, but it is much less pronounced than in Version 5 at epoch 100. The notebook records the best epoch for analysis but intentionally uses the final epoch-100 parameters for generation.

## Gradient and parameter checks

The tests verify finite gradients for all four trainable parameter matrices, matching gradient shapes, finite recurrent states, and that every parameter matrix actually changes during training.

```
embedding_gradient, input_gradient, recurrent_gradient, output_gradient = \
    tape.gradient(loss, [embedding_matrix, input_weights,
                        recurrent_weights, output_weights])
assert recurrent_gradient.shape == recurrent_weights.shape
assert not np.allclose(recurrent_weights.numpy(),
                       trained_recurrent_weights.numpy())
```

---

## Learned probabilities

After training, the model produces a next-character probability distribution from an ordered four-character context. For the context mode, the highest probabilities are:

```
space: 0.5088   l: 0.1632   i: 0.0957
a: 0.0596      s: 0.0581   t: 0.0311
```

---

## Neural text generation

For every prediction, the current four-character window is mapped to embeddings and processed from left to right. The hidden state starts from zeros for that window, the final state produces the logits, and a local NumPy generator samples the next character. The oldest context character is then removed and the window advances.

```
mode -> sampled character
ode? -> next sampled character
...
```

Five samples from the trained context mode with seed 42 are l, space, l, k, and space. The demonstration then generates 300 characters from the final parameters, and the same seed reproduces the same text.

---

## Included checks

The notebook assertions verify context-window construction, tensor and variable types, all recurrent parameter shapes, finite hidden states, data splits, finite losses, normalized probabilities, finite gradients, parameter changes, deterministic retraining, reproducible generation, requested output length, and the starting context.

A complete top-to-bottom run ends with the message `All checks passed.`

## What Version 6 teaches

The central progression is:

```
context -> IDs -> embeddings -> recurrent hidden states -> logits  
-> loss -> gradients -> update -> slide -> sample
```

Version 6 introduces recurrent sequence processing and the first hidden state in the project. The educational focus moves from a flattened fixed representation to an ordered computation that carries learned information from one context position to the next.

---

## Interpretation

The same four context embeddings are now processed sequentially. A shared recurrent matrix is reused at every position, while tanh introduces a nonlinear hidden transformation. The final 16-dimensional hidden state summarizes the ordered context before the output projection.

The model has 1032 parameters, slightly fewer than Version 5, so the architectural change is not simply an increase in model size. The training dynamics also change substantially: learning begins slowly and the best validation loss occurs much later, at epoch 92.

---

## Limitations

- the context still has a fixed length of four characters;
  - the hidden state starts from zeros for every training example;
  - generation also recomputes each four-character window from a zero hidden state instead of preserving state indefinitely;
  - the corpus is small and embedded directly in the notebook;
  - overlapping context windows can appear across the random training and validation split;
  - the recurrent unit is a simple tanh transformation with no gates or bias terms;
  - there is no early stopping or checkpoint restoration, so generation uses epoch-100 parameters even though validation is best at epoch 92;
  - generated text captures local sequential patterns but still has no long-term coherence.
- 

## Conclusion

Version 6 replaces the flattened context representation of Version 5 with explicit recurrent processing. Character IDs select trainable embeddings, the embeddings update a shared hidden state from left to right, and the final state predicts the next character.

The simple recurrent mechanism keeps the implementation inspectable while introducing ordered sequence processing, a nonlinear hidden state, and recurrent weights. It prepares the project for later versions using more advanced recurrent architectures such as GRU and LSTM networks.